Cowrie SSH Honeypot Lab
Justin Rodriguez

Objective
The goal of this lab was to deploy a Cowrie SSH honeypot, generate a controlled SSH session, and reconstruct the session from its logs. I wanted to verify that Cowrie could capture a connection, a login, commands entered in its simulated shell, and the disconnect. I then used the session ID to tie those events together.

Environment and Setup
I used Ubuntu through the Windows Subsystem for Linux (WSL) on my Windows PC. I installed Cowrie in a Python virtual environment to keep its dependencies separate from Ubuntu's system Python. After initializing Cowrie, I configured its SSH listener to use 127.0.0.1:2222 for local testing. I verified the listener using ss and confirmed that 127.0.0.1:2222 appeared in the Local Address:Port column. I then connected with the OpenSSH client using the test username labtest and a throwaway password.

Verification of the Cowrie SSH listener on the local interface



Troubleshooting
Cowrie initially failed to start, and the port check showed no listener on port 2222. Running Cowrie in the foreground revealed a duplicate listen_endpoints setting in the SSH configuration. After addressing that entry, a configuration check exposed a second duplicate, sftp_enabled, under the [shell] section. I backed up the configuration and replaced the working file with a minimal override for the local SSH listener. I verified that Cowrie could read the configuration, restarted it, and confirmed that the port was listening before attempting the test login.

SSH Session Analysis
I reviewed the JSON (JavaScript Object Notation) events in var/log/cowrie/cowrie.json and used the shared session ID ea31146690de to trace a single SSH connection from its initial connection through authentication, command activity, and disconnect. The timeline below omits the attempted password.

SSH Session Timeline

All times are in UTC. Each event shares the session ID ea31146690de.

1. 13:08:42.977 — Connection opened (cowrie.session.connect)
    a. The SSH client connected from 127.0.0.1:44870 to the Cowrie listener at 127.0.0.1:2222.
2. 13:09:26.736 — Login succeeded (cowrie.login.success)
    a. Cowrie accepted the labtest login and opened a simulated shell.
3. 13:11:20.163 — Username checked (cowrie.command.input)
    a. The test user entered whoami to check the username displayed by the simulated shell.
4. 13:11:20.166 — Working directory checked (cowrie.command.input)
    a. The test user entered pwd to check the current directory in the simulated shell.
5. 13:11:22.042 — Exit command entered (cowrie.command.input)
    a. The test user entered '*exit'* to leave the simulated shell.
6. 13:11:22.046 — Connection closed (cowrie.session.closed)
    a. Cowrie recorded the disconnect. The connection lasted about 159 seconds.

The login event identifies the test account, while the command events show what the user typed after logging in. Linking them by session ID makes it possible to attribute the commands to that specific connection.

Limitations

This lab used one controlled session from localhost. The results demonstrate that Cowrie captured the activity I generated and that I could reconstruct it from the logs. Testing a failed login or multiple simultaneous connections would give me more practice distinguishing sessions and comparing authentication outcomes.

Conclusion

I deployed a local SSH honeypot, resolved a configuration error that prevented startup, and verified that Cowrie captured the activity from my test session. The most useful part was tracing the session through the logs. By matching the session ID across events, I could determine when the connection opened, which account logged in, what commands were entered, and when the connection closed. This gave me hands-on practice turning individual log entries into a timeline an analyst can review.